Applied NLP

DATA 641

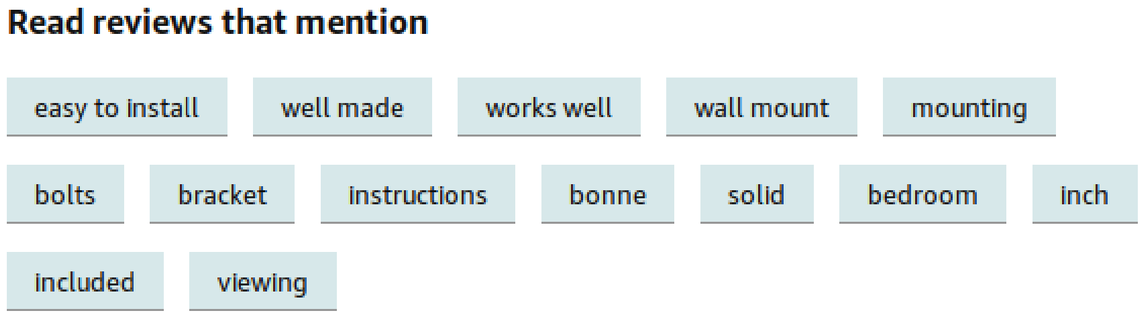
Lecture 10 — Keyphrase Extraction

*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly and Natural Language Processing in Action, Understanding, analyzing, and generating text with Python, Hobson Lane, Cole Howard, Hannes Hapke, First edition, MANNING

Keyphrase Extraction (KPE): Idea

- Consider a scenario where we want to buy a product, which has a hundred reviews, on Amazon. There is no way we are going to read all of them to get an idea of what users think about the product.

- To facilitate this, Amazon has a filtering feature: "Read reviews that mention." This presents a bunch of keywords or phrases that several people used in these reviews to filter the reviews.



Keyphrase Extraction: Approaches

KPE is a well-studied problem in the NLP community, and the two most commonly used methods to solve it are **supervised learning** and **unsupervised learning**.

- Supervised learning approaches require corpora with texts and their respective keyphrases and use engineered features or deep learning techniques. Creating such labeled datasets for KPE is a time and cost intensive endeavor.

- Unsupervised approaches that do not require a labeled dataset and are largely domain agnostic are more popular for KPE. These approaches are also more commonly used in real-world KPE applications. Recent research has also shown that state-of-the-art DL methods for KPE do not perform any better than unsupervised approaches
    - All the popular unsupervised KPE algorithms are based on the idea of representing the words and phrases in a text as nodes in a weighted graph where the weight indicates the importance of that keyphrase.
    - Keyphrases are then identified based on how connected they are with the rest of the graph.
    - The top-N important nodes from the graph are then returned as keyphrases. Important nodes are those words and phrases that are frequent enough and also well connected to different parts of the text.
    - The different graph-based KPE approaches differ in the way they select potential words/phrases from the text (from a large set of possible words and phrases in the entire text) and the way these words/phrases are scored in the graph.

Implementing KPE

In [2]:

```
import spacy
#!python -m spacy download en

nlp = spacy.load("en_core_web_sm")

mytext = open('nlphistory.txt').read()
```

In [3]:

```
#!pip install textacy
import textacy
from textacy import *
```

In [4]:

```
#convert the text into a spacy document
doc = textacy.make_spacy_doc(mytext, lang=nlp)
#doc
```

In [5]:
```
textacy.extract.keyterms.textrank(doc, topn=10)
```

Out[5]:
```
[('successful natural language processing system', 0.02464651768211853),
 ('statistical machine translation system', 0.024536501427749047),
 ('natural language system', 0.02043492712045758),
 ('statistical natural language processing', 0.0185001237620846),
 ('natural language task', 0.015740687358621126),
 ('machine learning algorithm', 0.015429377841496825),
 ('style machine learning method', 0.014884148969776667),
 ('term neural machine translation', 0.014711030542019226),
 ('speech recognition system', 0.01254550644897757),
 ('cache language model', 0.012536241051458 07)]
```

In [8]:
```
textacy.extract.keyterms.sgrank(doc, topn=10)
```

Out[8]:
```
[('natural language processing system', 0.39672753807910405),
 ('statistical machine translation', 0.03389673716977213),
 ('early', 0.0124560276469322),
 ('research', 0.012105591301842389),
 ('late 1980', 0.011317202097013824),
 ('real', 0.009228396482303705),
 ('world', 0.007518864639295327),
 ('example', 0.007263889822463248),
 ('statistical model', 0.007093222151328387),
 ('ELIZA', 0.006349659619718864)]
```

To address the issue of overlapping key phrases, `textacy` has a function: `aggregage_term_variants`.

Choosing one of the grouped terms per item will give us a list of non-overlapping key phrases!

In [10]:
```
terms = set([term for term,weight in textacy.extract.keyterms.sgrank(doc)])
print(textacy.extract.utils.aggregate_term_variants(terms))
```

```
[{'natural language processing system'}, {'statistical machine transla
tion'}, {'statistical model'}, {'late 1980'}, {'research'}, {'exampl
e'}, {'early'}, {'world'}, {'ELIZA'}, {'real'}]
```

Practical Advice

From a practical point of view, there are a few caveats to keep in mind when using such graph-based algorithms in production, though.

- The process of extracting potential n-grams and building the graph with them is sensitive to document length, which could be an issue in a production scenario. One approach to dealing with it is to not use the full text, but instead try using the first M% and the last N% of the text, since we would expect that the introductory and concluding parts of the text should cover the main summary of the text.

- Since each keyphrase is independently ranked, we sometimes end up seeing overlapping keyphrases (e.g., "buy back stock" and "buy back"). One solution for this could be to use some similarity measure (e.g., cosine similarity) between the topranked keyphrases and choose the ones that are most dissimilar to one another. `textacy` already implements a function to address this issue, as shown in the notebook.

- Seeing counterproductive patterns (e.g., a keyphrase that starts with a preposition when you do not want that) is another common problem. This is relatively straightforward to handle by tweaking the implementation code for the algorithm and explicitly encoding information about such unwanted word patterns.

- Improper text extraction can affect the rest of the KPE process, especially when dealing with formats such as PDF or scanned images. This is primarily because KPE is sensitive to sentence structure in the document. Hence, it is a good idea to add some post-processing to the extracted key phrases list to create a final, meaningful list without noise.

A custom solution could be a combination of an existing graph-based KPE algorithm that addresses the above-mentioned issues and a domain-specific list of heuristics, if available. From our experience, this covers the issues most commonly encountered with KPE in typical NLP projects.